Collision Detection

Collision Detection

1. Course Content
2. Preparation
3. Case Demonstration
4. Core Source Code Analysis

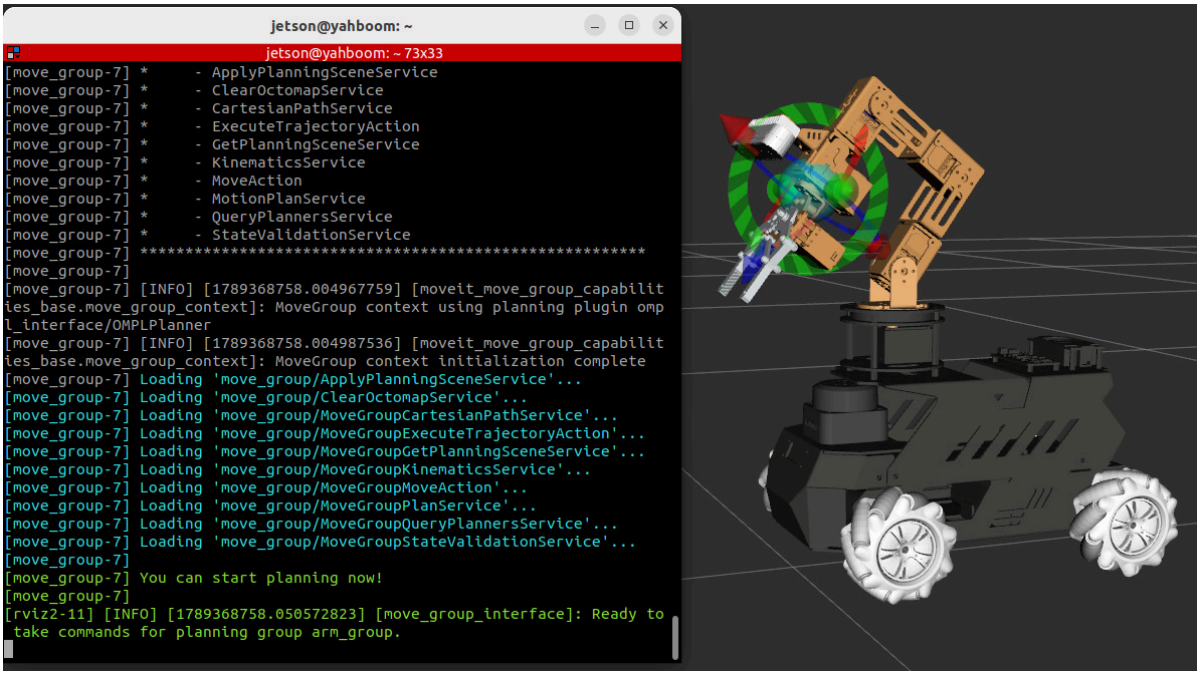
1. Course Content

This course explains how to use the MoveIt library to add obstacles in RViz and implement obstacle avoidance when MoveIt plans motion.

2. Preparation

```
ros2 launch yahboomcar_bringup moveit_simulation.launch.py sim2real:=true
rviz_config:=demo
```

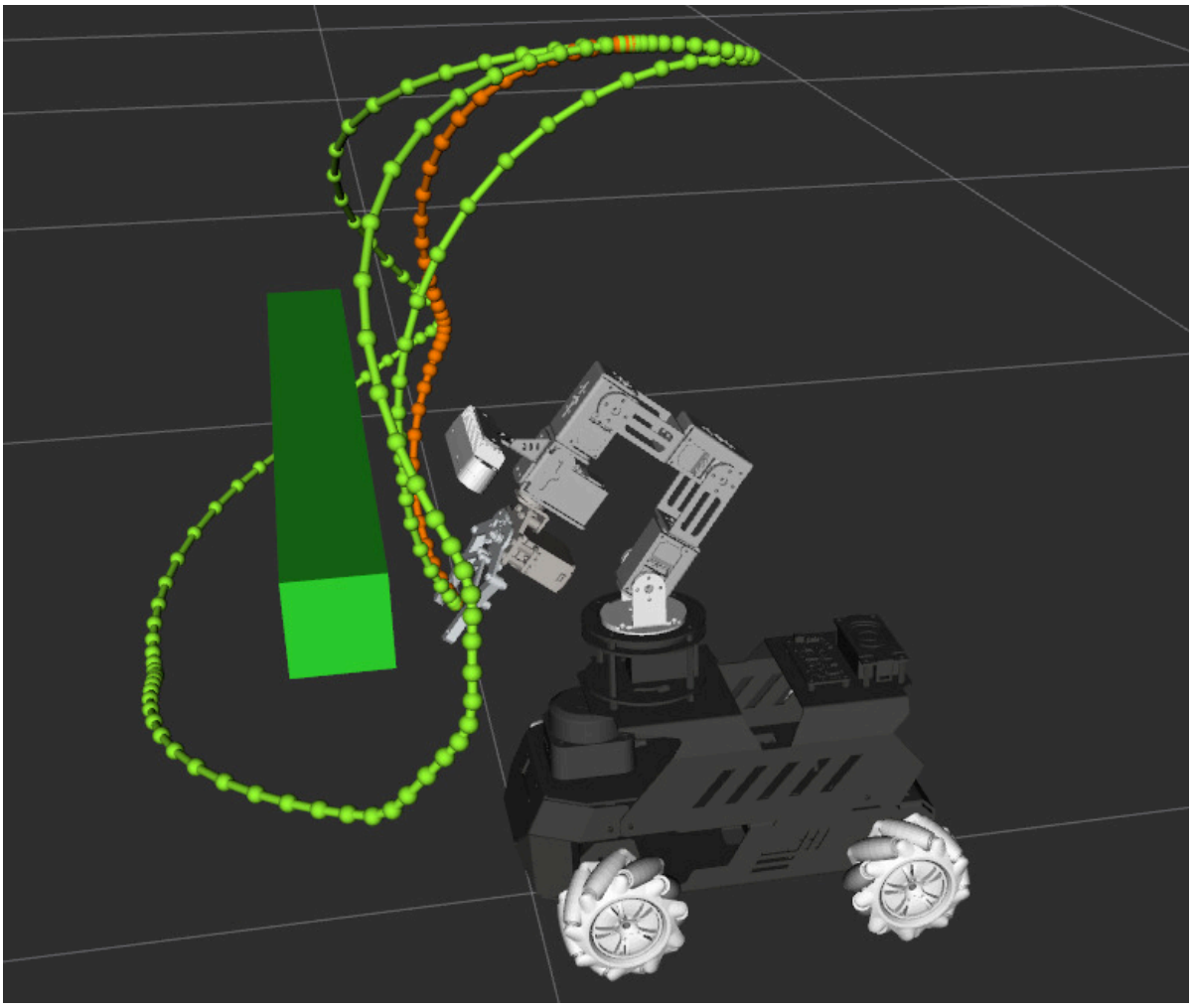
Once the program starts, when **You can start planning now!** appears in the terminal, it means the startup was successful, as shown in the figure below:



3. Case Demonstration

```
ros2 run moveit_demo obstacle_avoidance
```

- After the program runs, a rectangular obstacle will appear in the scene. The robot arm will move from the initial pose to above the obstacle, then below the obstacle, and finally return to the initial position. The robot arm will plan a motion trajectory to move around the obstacle (if a planned trajectory would collide with the obstacle, the collision position will be shown as a red sphere in RViz).



4. Core Source Code Analysis

- Source code path

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/moveit_demo/src/obstacle_avoidance.cpp
```

initialize function description:

- `MoveGroupInterface(shared_from_this(), "arm_group")`: Creates a MoveGroup control interface, bound to the `arm_group` planning group.
- `PlanningSceneInterface`: Creates a planning scene interface, used to add or remove collision objects in the scene.
- `setNumPlanningAttempts(10)` / `setPlanningTime(5.0)`: Sets the number of planning attempts and the maximum planning time.
- `setEndEffectorLink("gripper_tcp")` / `setPoseReferenceFrame("measure")`: Specifies the end link and the reference coordinate frame.
- `MoveItVisualTools(...)`: Creates an RViz visualization tool, used to draw trajectory lines.
- `getCurrentJointValues()`: Gets the current joint angle values of the robot arm, used to finally return to the initial position.

Add collision obstacle:

- `collision_object.header.frame_id = getPlanningFrame()`: Sets the reference coordinate frame of the collision object to the planning frame.
- `collision_object.id = "box"`: Sets the unique identifier of the collision object to "box".
- `primitive.type = primitive.BOX`: Sets the shape of the collision object to a cuboid.
- `primitive.dimensions[BOX_X/Y/Z]`: Sets the three side dimensions of the cuboid to 0.05 m, 0.05 m, and 0.5 m respectively (a tall, thin pillar-shaped obstacle).
- `box_pose.orientation.w = 0.7071, x = 0.7071`: Sets the pose of the collision object, rotating 90° about the X axis (in quaternion representation).
- `box_pose.position.x/y/z`: Sets the center position of the collision object to (0.11, 0.0, 0.33).
- `collision_object.operation = collision_object.ADD`: Sets the operation type to add.
- `addCollisionObjects(collision_objects)`: Adds the collision object to the planning scene; subsequent planning will automatically avoid it.

Named target motion (with retry):

- `targets = {"up", "down", "up"}`: Defines 3 named targets, moving to up, then down, then up in sequence.
- `setNamedTarget(targets[i])`: Sets the target to the preset named pose.
- `max_attempts = 5`: Each target tries planning up to 5 times; if it fails, it retries.
- `publishTrajectoryLine(...)`: Draws the motion trajectory line in cyan-green in RViz.
- `execute(plan)`: If planning succeeds, executes the trajectory.
- `executed`: Marks whether the current target was executed successfully; if not, an error is reported.

Return to the initial position:

- `setJointValueTarget(initial_joints)`: Sets the target to the recorded initial joint angles.
- `publishTrajectoryLine(..., rviz_visual_tools::ORANGE)`: Draws the return trajectory line in orange in RViz.
- `execute(plan_home)`: Executes the return trajectory, and the robot arm returns to the initial pose.
- `rclcpp::shutdown()`: Shuts down the node after the task is complete.

```
class Avoidance : public rclcpp::Node
{
public:
  Avoidance()
    : Node("obstacle_avoidance")
  {
    RCLCPP_INFO(this->get_logger(), "Initializing ObstacleAvoidance.");
  }

  void initialize()
  {
    // ===== 1. MoveGroupInterface =====
    move_group_interface_ =
      std::make_shared<moveit::planning_interface::MoveGroupInterface>(
        shared_from_this(), "arm_group");
```

```
    planning_scene_interface_ =
      std::make_shared<moveit::planning_interface::PlanningSceneInterface>();

    move_group_interface_->setNumPlanningAttempts(10);
    move_group_interface_->setPlanningTime(5.0);
    move_group_interface_->setMaxVelocityScalingFactor(1.0);
    move_group_interface_->setMaxAccelerationScalingFactor(1.0);
    move_group_interface_->setEndEffectorLink("gripper_tcp");
    move_group_interface_->setPoseReferenceFrame("measure");

    // ===== 2. Visual tools =====
    const moveit::core::JointModelGroup* joint_model_group =
      move_group_interface_->getRobotModel()->getJointModelGroup("arm_group");
    visual_tools_ = std::make_shared<moveit_visual_tools::MoveItVisualTools>(
      shared_from_this(), "measure", "obstacle_avoidance",
      move_group_interface_->getRobotModel());
    visual_tools_->deleteAllMarkers();
    visual_tools_->loadRemoteControl();

    // ===== 3. Record initial joint angles =====
    const std::vector<double> initial_joints =
      move_group_interface_->getCurrentJointValues();
    {
      const auto& jmg_names = joint_model_group->getVariableNames();
      RCLCPP_INFO(this->get_logger(),
        "=== Initial joint angles recorded ===");
      for (size_t i = 0; i < initial_joints.size() && i < jmg_names.size(); i++)
      {
        RCLCPP_INFO(this->get_logger(), "  %s = %.4f rad (%.2f deg)",
          jmg_names[i].c_str(), initial_joints[i],
          initial_joints[i] * 180.0 / M_PI);
      }
    }

    // ===== 4. Add collision object =====
    moveit_msgs::msg::CollisionObject collision_object;
    collision_object.header.frame_id = move_group_interface_->getPlanningFrame();
    collision_object.id = "box";

    shape_msgs::msg::SolidPrimitive primitive;
    primitive.type = primitive.BOX;
    primitive.dimensions.resize(3);
    primitive.dimensions[primitive.BOX_X] = 0.05;
    primitive.dimensions[primitive.BOX_Y] = 0.05;
    primitive.dimensions[primitive.BOX_Z] = 0.5;

    geometry_msgs::msg::Pose box_pose;
    box_pose.orientation.w = 0.7071;
    box_pose.orientation.x = 0.7071;
    box_pose.position.x = 0.11;
    box_pose.position.y = 0.0;
    box_pose.position.z = 0.33;

    collision_object.primitives.push_back(primitive);
    collision_object.primitive_poses.push_back(box_pose);
    collision_object.operation = collision_object.ADD;
```

```cpp
std::vector<moveit_msgs::msg::CollisionObject> collision_objects;
collision_objects.push_back(collision_object);

RCLCPP_INFO(this->get_logger(), "Add an object into the world");
planning_scene_interface_->addCollisionObjects(collision_objects);

// ===== 5. Move through named targets =====
const int max_attempts = 5;
const std::vector<std::string> targets = {"up", "down", "up"};

for (size_t i = 0; i < targets.size(); i++)
{
  RCLCPP_INFO(this->get_logger(),
    "=== Target %zu: '%s' ===", i + 1, targets[i].c_str());

  int attempt_count = 0;
  bool executed = false;

  while (attempt_count < max_attempts)
  {
    attempt_count++;
    move_group_interface_->setNamedTarget(targets[i]);

    moveit::planning_interface::MoveGroupInterface::Plan plan;
    bool success = (move_group_interface_->plan(plan) ==
                    moveit::core::MoveItErrorCode::SUCCESS);
    if (success)
    {
      // Visualize trajectory in RViz
      visual_tools_->publishTrajectoryLine(plan.trajectory_,
        move_group_interface_->getRobotModel()->getLinkModel("gripper_tcp"),
        joint_model_group, rviz_visual_tools::LIME_GREEN);
      visual_tools_->trigger();

      RCLCPP_INFO(this->get_logger(),
        "Target '%s' executing...", targets[i].c_str());
      move_group_interface_->execute(plan);
      executed = true;
      break;
    }
    else
    {
      RCLCPP_WARN(this->get_logger(),
        "Target '%s' planning attempt %d failed!",
        targets[i].c_str(), attempt_count);
    }
  }

  if (!executed)
  {
    RCLCPP_ERROR(this->get_logger(),
      "Target '%s' failed after %d attempts!",
      targets[i].c_str(), max_attempts);
  }
}
```

```
    // ===== 6. Return to initial joint angles =====
    {
      RCLCPP_INFO(this->get_logger(),
        "=== Returning to initial position: [%.4f, %.4f, %.4f, %.4f, %.4f] rad
===",
        initial_joints[0], initial_joints[1], initial_joints[2],
        initial_joints[3], initial_joints[4]);

      move_group_interface_->setJointValueTarget(initial_joints);

      moveit::planning_interface::MoveGroupInterface::Plan plan_home;
      if (move_group_interface_->plan(plan_home) !=
          moveit::core::MoveItErrorCode::SUCCESS)
      {
        RCLCPP_ERROR(this->get_logger(),
          "Return to initial position planning failed!");
      }
      else
      {
        // Visualize return trajectory in RViz
        visual_tools_->publishTrajectoryLine(plan_home.trajectory_,
          move_group_interface_->getRobotModel()->getLinkModel("gripper_tcp"),
          joint_model_group, rviz_visual_tools::ORANGE);
        visual_tools_->trigger();

        RCLCPP_INFO(this->get_logger(),
          "Executing return to initial position...");
        move_group_interface_->execute(plan_home);

        // Print actual final joint angles after return
        const auto& points = plan_home.trajectory_.joint_trajectory.points;
        if (!points.empty())
        {
          const auto& names = plan_home.trajectory_.joint_trajectory.joint_names;
          const auto& last = points.back();
          RCLCPP_INFO(this->get_logger(),
            "Return to initial position complete, joint angles:");
          for (size_t i = 0; i < names.size() && i < last.positions.size(); i++)
          {
            RCLCPP_INFO(this->get_logger(), "  %s = %.4f rad (%.2f deg)",
              names[i].c_str(), last.positions[i],
              last.positions[i] * 180.0 / M_PI);
          }
        }
      }
    }

    rclcpp::shutdown();
  }

private:
  std::shared_ptr<moveit::planning_interface::MoveGroupInterface>
move_group_interface_;
  std::shared_ptr<moveit::planning_interface::PlanningSceneInterface>
planning_scene_interface_;
```

```
  std::shared_ptr<moveit_visual_tools::MoveItVisualTools> visual_tools_;
};
```